

Centro Universitário Católica de Santa Catarina

Curso de Engenharia de Software

Disciplina: Segurança da Informação — Prof. Edson Vaz Lopes

Atividade Prática 01

Análise e melhoria de segurança em sistema web — Entrega final

Relatório técnico

Sistema de Ocorrências Acadêmicas (protótipo didático)

Integrantes do grupo:

Renato Colin Neto

Adrian Cesar

Gabriel Carvalho

Igor Thiago Seberino

Joinville (SC), maio de 2026

1. Identificação

1.1. Grupo

Integrantes:

- Renato Colin Neto
- Adrian Cesar
- Gabriel Carvalho
- Igor Thiago Seberino

1.2. Sistema analisado

Sistema de Ocorrências Acadêmicas — protótipo didático fornecido pelo professor, composto exclusivamente por front-end (HTML5, CSS3 e JavaScript) com persistência em localStorage do navegador. O sistema simula o registro, consulta e acompanhamento de ocorrências acadêmicas, com três perfis de usuário (ALUNO, PROFESSOR, ADMIN).

1.3. Repositório e publicação

Repositório do código entregue: <https://github.com/igorSeberino/web-security-analysis>

Sistema publicado (GitHub Pages): <https://igorSeberino.github.io/web-security-analysis/>

1.4. Versão de referência

Esta entrega consolida o trabalho da entrega parcial — apresentada anteriormente na disciplina — e adiciona as correções complementares necessárias para que o sistema possa ser utilizado em laboratório de forma minimamente segura. Toda a discussão a seguir refere-se ao código presente no arquivo `seguranca-da-informacao-AP01-main.zip` submetido nesta entrega final.

2. Descrição do sistema e regras de negócio

2.1. Funcionalidade

O sistema permite que usuários autenticados:

- Registrem novas ocorrências acadêmicas (nome do aluno, matrícula, tipo, prioridade, descrição e observação interna).
- Acrescentem dados de contato (CPF, e-mail, telefone) somente quando a ocorrência for do tipo "Solicitação administrativa" — princípio da minimização (LGPD).
- Alterem o status das ocorrências (Aberta, Em análise, Resolvida) conforme permissão.
- Excluam ocorrências (somente administradores).
- Exportem registros em JSON (escopo varia por perfil).
- Visualizem logs de auditoria armazenados localmente.

2.2. Perfis e regras

Três perfis com permissões distintas, implementadas no objeto PERMISSIONS do app.js:

- **ALUNO:** Cria ocorrências; visualiza listagem com PII mascarada; não vê observação interna; não exporta nem exclui.
- **PROFESSOR:** Cria e altera status; visualiza observação interna; exporta apenas ocorrências; PII permanece mascarada na exibição.
- **ADMIN:** Acesso completo — visualiza PII em claro, exporta ocorrências + logs, exclui registros, limpa logs, restaura dados e altera o próprio perfil ativo.

2.3. Arquitetura

O sistema é estritamente front-end. Não há servidor, banco de dados, API ou autenticação centralizada. Os arquivos entregues são:

- **index.html** — estrutura da interface, com Content-Security-Policy via meta-tag, banner de protótipo e fieldset condicional para dados de contato.
- **style.css** — estilização visual.
- **app.js** — lógica completa: autenticação, RBAC, sessão, validações, mascaramento, render, auditoria, throttling de login.
- **README.md** — instruções de execução, credenciais de demonstração e mapa dos controles aplicados.

3. Ativos identificados e classificação

A análise inicial do sistema (entrega parcial) identificou os ativos de informação a seguir, agora reclassificados conforme o nível de proteção requerido após as melhorias.

Ativo	Conteúdo	Classificação	Tratamento aplicado
Credenciais de usuário	E-mail e hash de senha (SHA-256)	Restrito	Hash no código; nunca em sessão/exportação
Dados pessoais (PII)	CPF, e-mail, telefone, nome do aluno	Confidencial	Mascaramento; coleta condicional; perfil ALUNO/PROFESSOR não vê em claro
Ocorrências acadêmicas	Tipo, prioridade, descrição, status	Interno	Visíveis a usuários autenticados conforme perfil
Observação interna	Notas administrativas vinculadas à ocorrência	Confidencial	Restrita para ALUNO
Sessão de usuário	id, name, email, role, classes, loggedAt	Interno	NUNCA contém senha ou hash; timeout 10 min
Logs de auditoria	Carimbo, usuário, perfil, ação, detalhe truncado	Interno	Retenção máxima 500 entradas; visíveis no painel
Exportações	Arquivo JSON com ocorrências e logs	Confidencial	Permitido a PROFESSOR (só ocorrências) e ADMIN (+ logs); nunca contém USERS/hashes

A classificação adota a convenção pública/interna/confidencial/restrita. Dados pessoais (PII) recebem cuidado adicional por estarem sob a Lei Geral de Proteção de Dados (LGPD).

4. Análise de riscos e controles aplicados

A tabela a seguir lista todos os problemas identificados na análise inicial, o risco associado, o controle aplicado nesta entrega e a justificativa técnica. A última linha aborda as limitações estruturais que permanecem em aberto.

Problema identificado	Risco associado	Controle / melhoria aplicada	Implementado?	Justificativa técnica
Senhas armazenadas em texto puro no array USERS	Comprometimento imediato de todas as contas a partir do código-fonte público	Substituição das senhas por hash SHA-256 (Web Crypto API)	Sim	Embora SHA-256 sem salt não seja adequado para produção, o código entregue não expõe mais credenciais em claro. A solução definitiva exige back-end com bcrypt/Argon2 e salt por usuário.
Token "FAKE_API_TOKEN" exposto no front-end	Falsa sensação de segurança e potencial abuso caso o sistema fosse integrado	Constante removida integralmente do app.js	Sim	Tokens secretos não podem existir em código front-end por princípio.
Credenciais pré-preenchidas nos inputs do HTML	Qualquer visitante logava no primeiro clique	Atributos value=... removidos; campos vazios com autocomplete=off	Sim	Tela de login agora exige entrada efetiva e não revela usuário/senha por padrão.
Lista pública de usuários e senhas visível na tela de login	Exposição direta de credenciais a qualquer pessoa que acesse o site	Bloco movido para <details> colapsado, com aviso explícito de uso didático	Sim	O acesso à lista ainda é possível para fins de demonstração, mas exige ação consciente do usuário.
Sessão salvava o objeto USER inteiro (incluindo senha) no localStorage	Senha ficava persistida em claro no navegador após o login	saveSession() agora monta um objeto seguro com apenas { id, name, email, role, classes, loggedAt }	Sim	Não há mais credenciais persistidas em localStorage. Em produção, sessão seria controlada por servidor via JWT/cookie HttpOnly.
Ausência de proteção contra força bruta no login	Brute-force trivial das três contas conhecidas	Throttling: 5 tentativas falhas bloqueiam o login por 5 minutos; mensagens genéricas	Sim	Throttling client-side é best-effort (pode ser burlado por DevTools), mas constitui camada inicial. Solução completa exige rate-limit server-side.
Permissão livre de troca de perfil (roleSelect aberto)	Escalonamento trivial para ADMIN por	changeRole() agora exige permissão; seletor oculto para	Sim	Atende ao princípio do menor privilégio.

Problema identificado	Risco associado	Controle / melhoria aplicada	Implementado?	Justificativa técnica
	qualquer usuário	não-ADMIN		
Ausência de RBAC nos handlers críticos	Qualquer usuário podia executar funções administrativas chamando-as via console	Mapa PERMISSIONS consultado por hasPermission() / requirePermission() em todo handler sensível	Sim	Centraliza autorização. Em back-end, esta validação deveria reocorrer no servidor.
Vulnerabilidade XSS na renderização da tabela (innerHTML sem escape)	Execução de scripts maliciosos vinda de campos de entrada	Funções escapeHTML() e sanitizeInput() aplicadas em toda saída dinâmica	Sim	Mitiga XSS refletido/armazenado. A CSP via meta-tag adiciona defesa em profundidade.
onclick inline nos botões da tabela com IDs concatenados	Vetor residual de XSS se um ID escapasse pela validação	Botões usam data-act / data-id e event delegation no JS	Sim	Elimina injeção via atributo, alinhado a boas práticas modernas (sem inline event handlers).
Dados pessoais (CPF/e-mail/telefone) visíveis a todos os perfis	Violação de privacidade e da LGPD (princípio da minimização)	Mascaramento (maskCPF, maskEmail, maskPhone); revelação apenas para ADMIN; campos condicionais no formulário	Sim	PII só é coletada quando necessária e só é exibida em claro com permissão. Acesso é logado.
Observação interna visível ao ALUNO	Vazamento de informação administrativa	Renderização exibe "Restrito" para perfis sem permissão seeInternalNote	Sim	Separação de visibilidade por papel.
Busca varrendo todos os campos (CPF, observação interna)	Aluno conseguia inferir dados mascarados por substring	Busca filtrada para nome, matrícula, tipo, prioridade e status	Sim	Mascaramento deixa de ser contornável pela funcionalidade de busca.
Sem timeout de sessão	Sessões abandonadas em máquinas compartilhadas	Timeout de 10 minutos com renovação por atividade (click, mousemove, keydown, touchstart)	Sim	Reduz risco de uso indevido após inatividade.
Sem validação de formulário	Entrada de dados inválidos/maliciosos	Validações: tamanhos mínimos, regex para e-mail, dígitos para CPF/telefone, declaração LGPD obrigatória	Sim	Garante integridade básica dos dados.

Problema identificado	Risco associado	Controle / melhoria aplicada	Implementado?	Justificativa técnica
Logs detalhados expondo PII e crescimento ilimitado	Logs viravam fonte secundária de vazamento	Logs reduzidos a metadados; detalhes truncados em 240 caracteres; retenção máxima de 500 entradas	Sim	Reduz superfície de exposição e custo de armazenamento.
Exportação incluía USERS, token e localStorage cru	Vazamento de credenciais e tokens em download	Exportação restrita por perfil: ADMIN = ocorrências + logs; PROFESSOR = ocorrências; ALUNO = nada. NUNCA inclui USERS ou hashes	Sim	Princípio do menor privilégio aplicado à exportação.
Botões destrutivos disponíveis a todos	Perda acidental ou maliciosa de dados	Botões escondidos sem permissão + confirm() obrigatório + log de auditoria em cada ação destrutiva	Sim	Confirmação explícita + rastreabilidade.
Ausência de cabeçalhos defensivos	Risco de SOP/scripts externos indevidos	CSP via meta-tag; referrer no-referrer; robots noindex,nofollow	Sim	Não substitui cabeçalhos HTTP reais (que exigiriam servidor), mas adiciona camada base.
Banco de dados centralizado, sessões assinadas, logs imutáveis	Manipulação por DevTools, perda de evidência, falsificação de sessão	Não implementável em front-end puro	Não (dependência de back-end)	Discutido na seção 6 — soluções exigem servidor, banco protegido e SIEM.

5. Melhorias implementadas — visão detalhada

As melhorias agrupam-se em sete famílias de controles. A primeira parte resume os controles já entregues na entrega parcial; a segunda detalha as correções adicionadas para tornar o sistema utilizável.

5.1. Controles consolidados da entrega parcial

5.1.1. RBAC (Role-Based Access Control)

Mapa PERMISSIONS define oito capacidades por perfil (create, view, changeStatus, delete, export, reset, clearLogs, changeRole, revealSensitive, seeInternalNote, seeAllOccurrences). Funções hasPermission() e requirePermission() são chamadas no início de cada handler crítico; em caso de negação, a ação é interrompida e um evento ACESSO_NEGADO é registrado no log.

5.1.2. Sanitização e proteção contra XSS

Toda saída dinâmica passa por escapeHTML(), que converte os caracteres &, <, >, ", ', ` . As entradas de formulário são adicionalmente normalizadas por sanitizeInput(), que remove caracteres de controle e tags HTML.

5.1.3. Mascaramento de PII

Funções maskCPF, maskEmail e maskPhone produzem versões parciais dos dados para exibição. A versão em claro só é apresentada quando o perfil ativo possui revealSensitive: true (apenas ADMIN). A revelação pode ser auditada nos logs do sistema.

5.1.4. Timeout de sessão

A constante SESSION_TIMEOUT_MS define 10 minutos de inatividade. Os eventos click, mousemove, keydown e touchstart reiniciam o cronômetro. O evento SESSAO_EXPIRADA é registrado quando a sessão é encerrada automaticamente.

5.1.5. Validações e confirmações

createOccurrence() valida: nome com mínimo 3 caracteres, matrícula com mínimo 4 caracteres, descrição com mínimo 10 caracteres e declaração LGPD obrigatória. Em "Solicitação administrativa", CPF, e-mail e telefone também são validados (regex de e-mail e contagem de dígitos). Ações destrutivas (delete, clearLogs, resetData) exigem confirm() explícito.

5.1.6. Logs e feedback

Cada ação relevante grava em log: timestamp, e-mail, perfil, ação, detalhe e alvo (quando aplicável). Mensagens ao usuário usam showMessage() (sucesso/erro/informativo) em vez de alert() bloqueante.

5.2. Correções adicionadas na entrega final

5.2.1. Eliminação de credenciais em texto puro

Senhas no array USERS substituídas por hash SHA-256. O login calcula o hash da senha digitada via crypto.subtle.digest e compara com o valor armazenado. As três senhas demonstrativas são: Aluno@2026, Prof@2026 e Admin@2026. A constante FAKE_API_TOKEN foi removida integralmente do código.

5.2.2. Tela de login limpa

Inputs do formulário não mais carregam value pré-preenchido. A lista de usuários de demonstração foi movida para um elemento <details> colapsado, com aviso explícito de que se trata de credenciais didáticas e de que as senhas estão armazenadas como hash.

5.2.3. Sessão sem credenciais

A função saveSession() agora produz um objeto seguro contendo apenas dados de exibição (id, name, email, role, classes, loggedAt). Hash, senha e qualquer outra credencial são deliberadamente excluídos. O localStorage do navegador deixa de armazenar qualquer material sensível.

5.2.4. Throttling de login

Após 5 tentativas falhas consecutivas, o sistema bloqueia novas tentativas por 5 minutos. O estado é persistido em uma chave separada (ocorrencias_login_tentativas) e os eventos LOGIN_FALHOU e LOGIN_BLOQUEADO são registrados. A mensagem de erro é genérica ("Credenciais inválidas"), evitando enumeração de usuários.

5.2.5. Eliminação de onclick inline

Os botões "Em análise", "Resolver" e "Excluir" da tabela usam atributos data-act e data-id; o tratamento é centralizado por delegação no listener occurrencesTable.click. Isto elimina o vetor residual de XSS por injeção em atributos.

5.2.6. Busca com escopo reduzido

A busca agora percorre exclusivamente nome, matrícula, tipo, prioridade e status. CPF, e-mail, telefone e observação interna estão fora do escopo de busca para que o mascaramento não seja contornável.

5.2.7. Logs limitados e exportação por perfil

Logs têm retenção máxima de 500 entradas e os detalhes são truncados em 240 caracteres. A exportação produz JSON com objeto { exportedAt, exportedBy, role, occurrences } para PROFESSOR e adicionalmente { logs } para ADMIN. O array USERS e os hashes jamais são incluídos.

5.2.8. Defesa em camadas no HTML

A tag meta http-equiv="Content-Security-Policy" restringe scripts e estilos à própria origem; meta name="referrer" define no-referrer; meta name="robots" define noindex,nofollow. O sistema exibe banner permanente declarando seu caráter de protótipo.

5.2.9. Minimização de coleta (LGPD)

CPF, e-mail e telefone são solicitados apenas quando a ocorrência é classificada como "Solicitação administrativa". Para os demais tipos, esses campos sequer são exibidos no formulário.

6. Limitações remanescentes

Mesmo após as melhorias, o sistema permanece um protótipo front-end. As limitações abaixo são estruturais e exigem componentes de back-end e infraestrutura para serem efetivamente endereçadas:

Limitação atual	Por que persiste em front-end puro	Solução em produção
Hashes inspecionáveis no código	Qualquer usuário lê o app.js publicamente	Hash no servidor (bcrypt/Argon2) + base de credenciais protegida
RBAC contornável via DevTools	Usuário pode editar PERMISSIONS no console e salvar sessão arbitrária	Decisão de autorização exclusivamente no servidor (middleware ou policy engine)
Sessão sem assinatura	Sem servidor para emitir e validar tokens	JWT assinado + cookie HttpOnly + Secure + SameSite + proteção CSRF
Throttling burlável	Estado vive no localStorage	Rate limit no servidor + CAPTCHA + bloqueio por IP/conta
Logs locais sem integridade	Podem ser apagados ou modificados	Pipeline de auditoria centralizado (SIEM: Elastic Stack, Splunk, Graylog)
localStorage manipulável	Não há transação ou validação centralizada	Banco de dados relacional/documental com transações (PostgreSQL, MongoDB)
Sem cabeçalhos HTTP reais	CSP por meta-tag tem alcance limitado	Servidor entrega Content-Security-Policy, Strict-Transport-Security, X-Frame-Options, X-Content-Type-Options, Permissions-Policy
Sem proteção contra adulteração de cliente	JS roda integralmente no navegador	Validação reaplicada no servidor para toda operação relevante
Sem backup ou redundância	localStorage é local	Backup automatizado, replicação e plano de recuperação

Reforça-se que estas limitações não invalidam o exercício didático: o sistema atende aos objetivos pedagógicos da disciplina de demonstrar quais defesas são possíveis apenas no front-end e quais exigem arquitetura back-end completa.

7. Aderência a princípios da LGPD

Embora se trate de protótipo didático sem coleta real de dados, os controles implementados estão alinhados aos seguintes princípios da Lei nº 13.709/2018:

- **Finalidade e adequação:** banner de protótipo explicita que o uso é apenas didático e não deve operar com dados reais.
- **Necessidade (minimização):** CPF, e-mail e telefone só são coletados em ocorrências do tipo "Solicitação administrativa".
- **Livre acesso e transparência:** a declaração de base legal é obrigatória no formulário (checkbox privacyAck).
- **Segurança:** RBAC, mascaramento, hash de senhas, timeout de sessão, throttling e auditoria.
- **Prevenção:** sanitização, escape de HTML, validações e CSP.
- **Não discriminação:** permissões baseadas em papel funcional, não em atributos pessoais.
- **Responsabilização e prestação de contas:** logs persistem cada ação significativa com perfil e timestamp.

Para operação produtiva, seriam ainda necessários: relatório de impacto à proteção de dados (RIPD), política de retenção, mecanismos de exercício dos direitos do titular (art. 18) e Encarregado de Proteção de Dados (DPO).

8. Plano de evolução para produção

Para que o sistema possa ser utilizado em ambiente produtivo, recomenda-se a seguinte ordem de evolução:

Fase 1 — Back-end básico

- API REST com Node.js/Express ou Java Spring Boot.
- Banco relacional (PostgreSQL ou MySQL) com modelo: users, occurrences, audit_logs.
- Hash de senhas com Argon2id ou bcrypt + salt único por usuário.
- Autenticação por JWT assinado, com refresh tokens armazenados em cookie HttpOnly + Secure + SameSite=Strict.
- Reaplicação de TODAS as validações e regras de autorização no servidor; front-end passa a ser apenas camada de apresentação.

Fase 2 — Endurecimento

- Cabeçalhos HTTP de segurança via servidor: CSP rigorosa, HSTS, X-Frame-Options, X-Content-Type-Options.
- Rate limiting real (express-rate-limit, Cloudflare ou WAF) e CAPTCHA na tela de login.
- Migração para criptografia de dados em repouso para PII (transparent data encryption no banco).
- Pipeline de auditoria com SIEM (Elastic Stack/Splunk/Graylog) e logs append-only.
- Backups automatizados + plano de recuperação testado.

Fase 3 — Governança

- Adoção de OAuth2/OpenID Connect para SSO com o provedor da instituição.
- Política formal de senhas e MFA para perfis administrativos.
- Relatório de Impacto à Proteção de Dados (RIPD) e nomeação de DPO.
- Programa de revisão periódica de acessos (acesso por necessidade comprovada).
- Testes regulares: pentest, SAST e DAST integrados ao pipeline CI/CD.

9. Conclusão

Esta entrega final consolida o trabalho da entrega parcial e adiciona as correções complementares necessárias para que o sistema seja minimamente utilizável em ambiente didático. O resultado é um protótipo no qual:

- Não há credenciais em texto puro no código, na sessão ou na exportação.
- Toda ação sensível passa por verificação de permissão e gera registro de auditoria.
- Dados pessoais são coletados apenas quando necessários e exibidos com mascaramento.
- A interface está protegida contra XSS por escape, sanitização e CSP.
- A tela de login tem proteção básica contra força bruta e não expõe usuários ou senhas por padrão.
- As sessões expiram por inatividade.

Apesar desses ganhos, é fundamental destacar que o sistema permanece estritamente um exercício didático. As limitações estruturais — possibilidade de manipulação via DevTools, ausência de back-end, dependência exclusiva do navegador para armazenamento e auditoria — impedem que o protótipo seja considerado seguro para produção. Sua utilidade real está em demonstrar, de forma prática, quais defesas o front-end consegue oferecer e onde exatamente a arquitetura back-end se torna indispensável.

O grupo entende, portanto, que o objetivo pedagógico da Atividade Prática 01 — analisar um sistema vulnerável, propor melhorias e implementá-las dentro de uma restrição arquitetural deliberada — foi alcançado. O sistema entregue está pronto para uso em laboratório, com a ressalva clara, registrada no banner da própria interface e neste relatório, de que não deve receber dados pessoais reais.

9.1. Checklist da entrega

- ✓ Código alterado entregue (ZIP `seguranca-da-informacao-AP01-main.zip`).
- ✓ Relatório técnico (este documento, em PDF).
- ✓ Link do sistema publicado em GitHub Pages: <https://igorSeberino.github.io/web-security-analysis/>
- ✓ Análise de riscos com tabela problema/risco/controle/justificativa (seção 4).
- ✓ Identificação e classificação dos ativos (seção 3).
- ✓ Discussão de melhorias dependentes de back-end (seção 6).
- ✓ Aderência a princípios da LGPD (seção 7).
- ✓ Conclusão argumentando se o sistema pode/não pode ir para produção (seção 9).